Flight finder: navigating your air travel options

1. Introduction

Flight Finder is a centralized platform designed to simplify air travel planning. By aggregating real-time flight data, the application allows users to search, compare, and track various flight options across multiple airlines, helping them find the most cost-effective and time-efficient routes.

- **Project Title:** flightfinder: navigating your air travel options.
- **Team Members:**

Team Leader : Majji Dinesh Dinesh
Team member : Adilakshmi Yadagiri
Team member : Harini Vempadapu
Team member : Hemanth Venkata Sai Vanacharla

2. Project Overview

The goal of this project is to solve the complexity of fragmented travel data.

- Problem: Travelers jump between multiple tabs to compare prices and schedules.
- Solution: A unified dashboard that fetches real-time API data, provides intelligent filtering (price, duration, layovers), and manages user bookings.
- Core Stack: React.js (Frontend), Node.js/Express (Backend), MongoDB (Database), and Amadeus/Skyscanner API (Data Source).

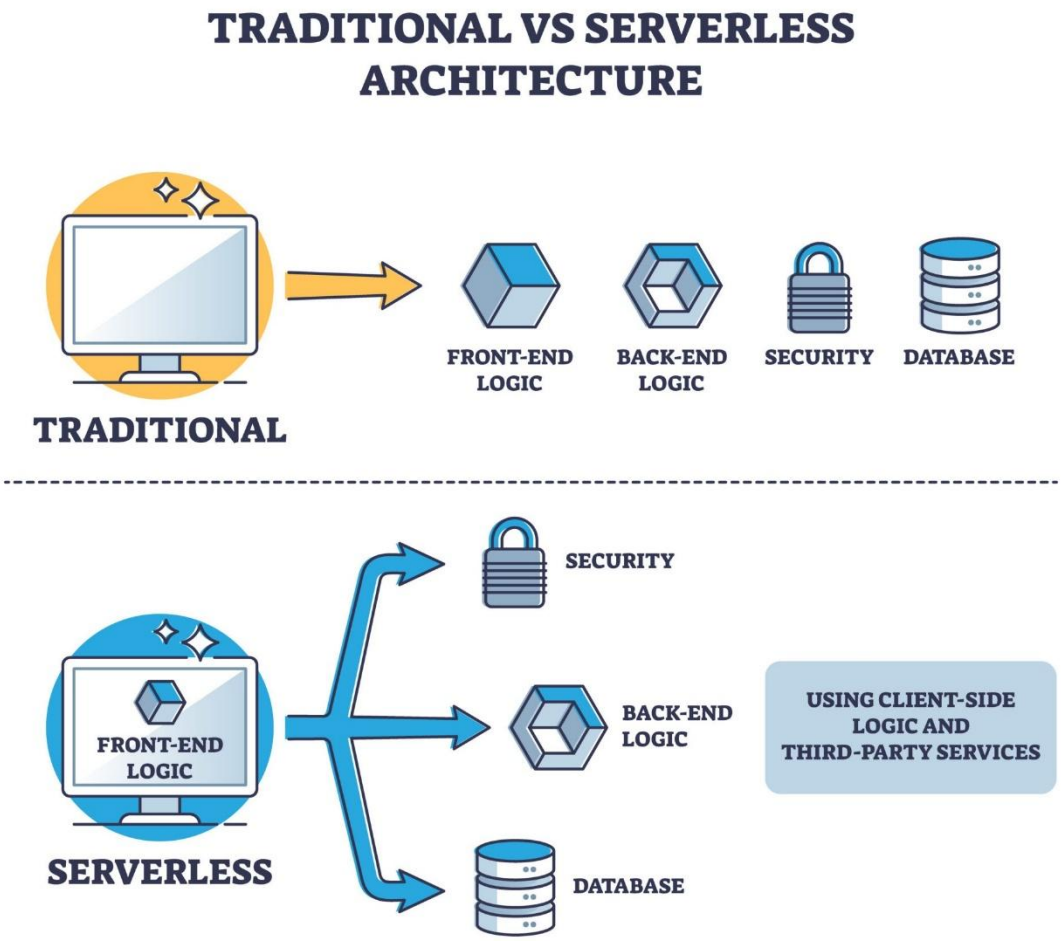
3. Architecture

The application follows a **Decoupled Client-Server Architecture**.

Frontend: A Single Page Application (SPA) built with React for a seamless UI.

Backend: A RESTful API built with Node.js to handle business logic and external API proxying.

External Layer: Integration with Third-party Flight APIs and OAuth providers.



4. Setup Instructions

To run this project locally, ensure you have **Node.js** and **npm** installed.

1. **Clone the repository:** git clone https://github.com/youruser/flight-finder.git
2. **Install Backend dependencies:** cd backend && npm install
3. **Install Frontend dependencies:** cd ../frontend && npm install
4. **Environment Variables:** Create a .env file in the root of the /backend folder:
    - PORT=5000
    - MONGO_URI=your_mongodb_link

- FLIGHT_API_KEY=your_api_key
- JWT_SECRET=your_secret_key

5. Folder Structure

```
flight-finder/
├── backend/
│   ├── controllers/   # Route logic
│   ├── models/        # Database schemas
│   ├── routes/        # API endpoints
│   └── server.js      # Entry point
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/ # Reusable UI
│   │   ├── pages/      # Search, Results, Profile
│   │   └── services/   # API calling logic
└── package.json
```

6. Running the Application

- **Start Backend:** In the /backend folder, run npm run dev.
- **Start Frontend:** In the /frontend folder, run npm start.
- The app will be accessible at **http://localhost:3000**.

7. API Documentation

- Document all endpoints exposed by the backend.

• Include request methods, parameters, and example responses.

8. Authentication

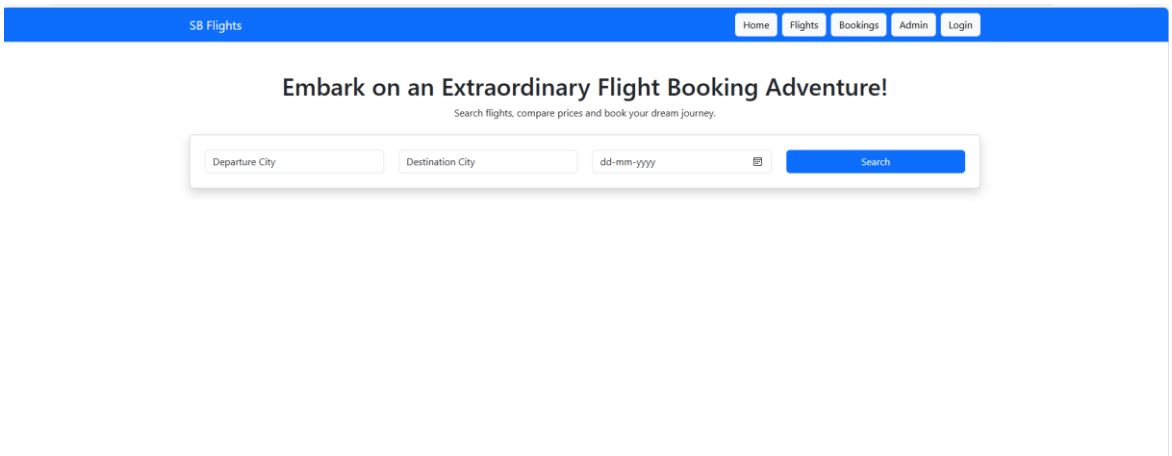
The system uses **JWT (JSON Web Tokens)** for secure access.

- **Registration/Login:** Handled via Bcrypt hashing for password security.
- **Protected Routes:** Bookings and Profile pages require a valid token in the Authorization Header: Bearer <token>.

9. User Interface

The UI is designed with a Mobile-First approach using Tailwind CSS.

- **Search Bar: Dynamic autocomplete for airport codes.**
- **Results Grid: Sortable cards showing airline logos, prices, and flight duration.**
- **Filter Sidebar: Adjustable sliders for price range and flight times.**

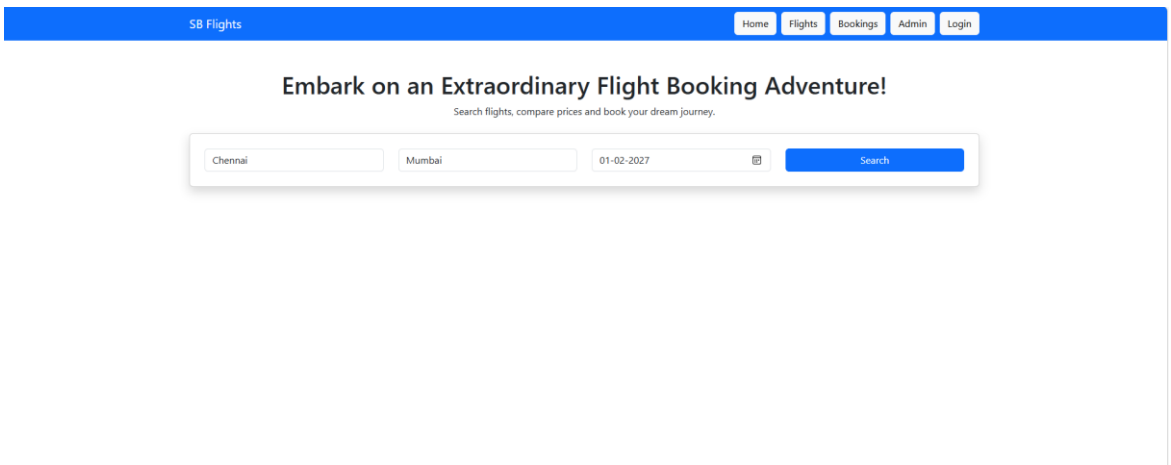


10. Testing

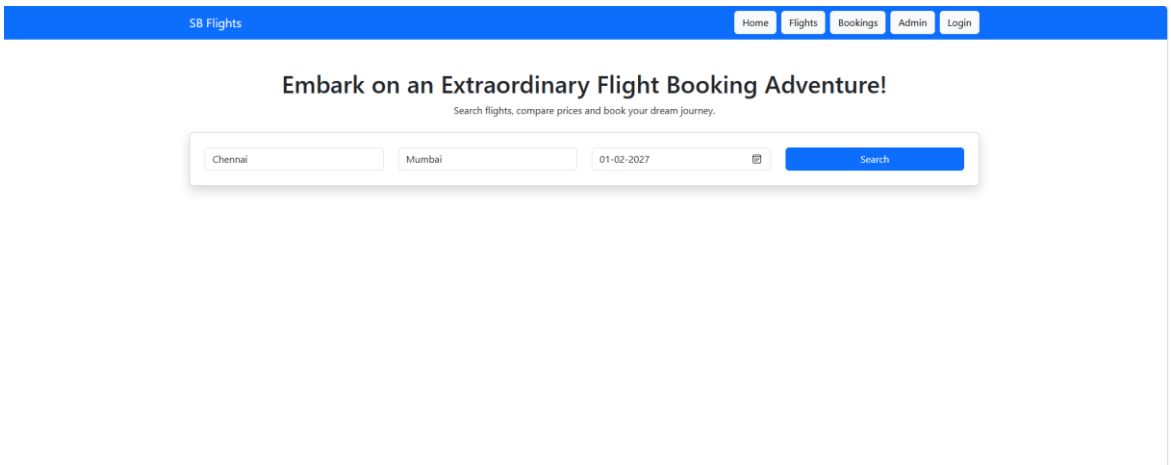
- **Unit Testing: Performed using Jest for backend logic.**
- **Component Testing: React Testing Library ensures UI elements render correctly.**
- **Integration Testing: Postman is used to verify API response formats and error codes (404, 500).**

11. Screenshots or Demo

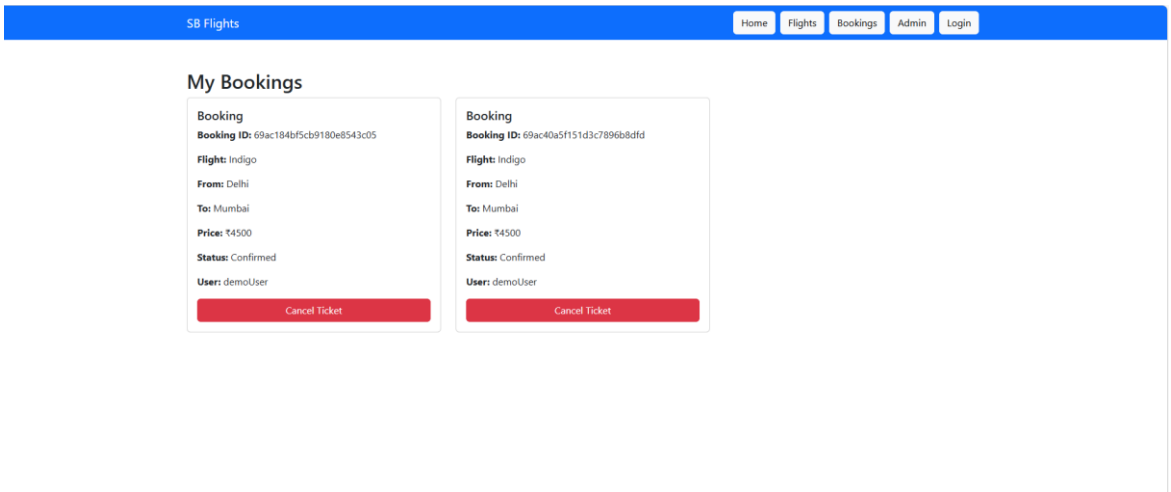
Searching Flights:



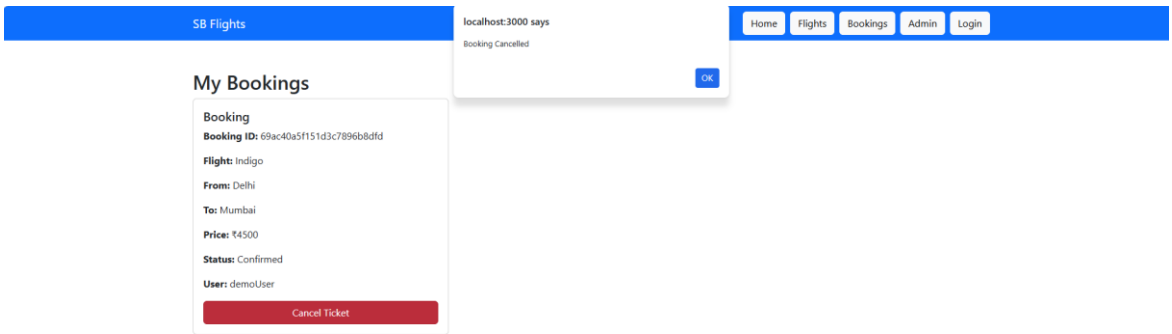
Available Flights:



Booking Flights:



Cancel Flights:



12. Known Issues

- API Latency: External flight data may take 3-5 seconds to populate during peak hours.
- Mobile Keyboard: On some smaller Android devices, the search input may be partially obscured by the soft keyboard.

13. Future Enhancements

- **Price Prediction:** Using Machine Learning to suggest whether a user should "Buy Now" or "Wait" for a lower price.
- **Multi-City Booking:** Expanding the search logic to include multi-stop itineraries.
- **Email Alerts:** Automated notifications when a tracked flight drops in price.